

End-to-End Implementation of a Neuro-Evolutionary Autonomous Navigational System: A Capstone Project in Applied AI

Deeghayu Suwahas Adhikari, 258722D, MDS&AI, UoM

Abstract—This capstone project presents the end-to-end design, implementation, and evaluation of an autonomous vehicle controller utilizing a neuro-evolutionary framework. A primary objective of this research was to eschew high-level, "black-box" deep learning libraries (such as TensorFlow or PyTorch) to demonstrate a profound mastery of the underlying mathematics. Instead, the entire Artificial Intelligence pipeline—including a custom Multi-Layer Perceptron (MLP) and a Genetic Algorithm (GA) optimizer—was engineered from scratch using foundational principles. The system integrates a custom-built virtual LiDAR perception array with a handcrafted kinematic physics engine to simulate a highly dynamic environment. By utilizing stochastic search in the synaptic weight space, the framework successfully navigates non-differentiable reward structures where traditional gradient-based learning struggles. The resulting system successfully evolved autonomous agents capable of complex obstacle avoidance, precise steering, and dynamic traffic navigation within 50 generations.

Index Terms—Neuro-Evolution, Genetic Algorithms, Deep Learning, Multi-Layer Perceptron, Autonomous Vehicles, Kinematic Modeling, Applied AI.

I. INTRODUCTION

THE development of autonomous navigational systems represents one of the most complex challenges in modern computational intelligence, requiring a seamless, real-time integration of perception, cognition, and actuation. While deep reinforcement learning has shown immense promise in this domain, the implementation of such systems often relies on highly abstracted frameworks. These "black-box" APIs obscure the underlying linear algebra, matrix transformations, and optimization dynamics that drive the cognitive engine.

The core motivation of this capstone project is to demystify these complex architectures by engineering a complete autonomous driving simulation entirely from scratch. Rather than utilizing pre-existing game engines (like Unity or Unreal) or machine learning libraries, every component of this environment was programmed from the ground up. This includes the 2D rendering loop, the collision detection algorithms, the physical kinematics of the vehicle, the neural network topology, and the evolutionary optimization logic.

By building the system from first principles, this project rigorously investigates the raw mechanics of Connectionist

Artificial Intelligence and the efficacy of evolutionary computing—specifically Genetic Algorithms (GA)—in optimizing a feedforward neural network in environments with sparse, non-differentiable rewards.

II. CORE ALGORITHMIC IMPLEMENTATION

The decision to build the system without external machine learning dependencies required a granular approach to algorithm design. The architecture is bifurcated into a high-performance simulation engine (handling physics and perception) and a mathematical core (the AI Controller).

A. Deep Learning from First Principles

The cognitive engine of the simulation relies on a custom implementation of a Multi-Layer Perceptron (MLP). The forward propagation mechanics—comprising matrix dot products, bias additions, and non-linear activation transformations—were hand-coded. This approach guarantees absolute control over the computational graph and allows for real-time visualization of synaptic firing. It demonstrates a foundational understanding of how data flows through a neural topology to produce deterministic actuation commands.

B. Gradient-Free Optimization

Autonomous navigation is inherently a sequential decision-making problem characterized by sparse rewards (e.g., a binary outcome of surviving versus crashing). In such environments, traditional backpropagation via gradient descent is often ineffective without complex value-function approximations (like Q-learning). To overcome this, a Genetic Algorithm was implemented. This approach treats the weights and biases of the neural network as an actionable genome, performing heuristic global searches across the high-dimensional weight space to evolve optimal driving behaviors over successive generations.

III. SIMULATION PHYSICS AND PERCEPTION

A. Custom Kinematic Physics Engine

To ensure the simulation poses a biologically and physically plausible challenge for the AI, a kinematic bicycle model was implemented. Unlike simplistic point-mass models that can pivot instantly, this engine accounts for forward momentum,

D. S. Adhikari (258722D) is a candidate in the Master of Data Science & Artificial Intelligence (MDS&AI) program at the Faculty of Information Technology, University of Moratuwa, Sri Lanka. (e-mail: adhikari-amnds.25@uom.lk).

This report is submitted in partial fulfillment of the requirements for the module IN25-S3-CS5998: Capstone Project (Final Report).

lateral friction, and realistic turning radii. The differential positional updates per frame are defined as:

$$x_{t+1} = x_t - \sin(\theta_t) \cdot v_t \quad (1)$$

$$y_{t+1} = y_t - \cos(\theta_t) \cdot v_t \quad (2)$$

where v_t is the velocity subject to a constant acceleration and a friction coefficient $C_f = 0.05$. Crucially, the rotational heading θ is mathematically coupled to the current velocity; the physics engine strictly prohibits the vehicle from rotating when $v_t = 0$, forcing the neural network to learn the relationship between speed and steering angle.

B. LiDAR Raycasting and Spatial Awareness

Spatial awareness is simulated through a handcrafted array of $n = 5$ virtual LiDAR sensors radiating from the vehicle's front chassis. To determine the proximity of track boundaries and dynamic traffic, highly optimized line-segment intersection algorithms were programmed from scratch. The exact point of intersection and the normalized distance scalar t are calculated using the standard intersection formulas:

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)} \quad (3)$$

By bounding $t \in [0, 1]$, the engine accurately detects the closest physical obstacle along each ray. These inverted distance values $(1 - t)$ act as the direct, normalized input features for the neural network.

IV. NEURAL ARCHITECTURE AND EVOLUTION

A. Manual Matrix Operations in the MLP

The autonomous controller utilizes a three-tier MLP topology: an input layer (5 neurons for LiDAR), a fully connected hidden layer (6 neurons), and an output layer (4 neurons corresponding to Forward, Reverse, Left, Right).

The pre-activation z and activation a for any given neuron j in layer l were implemented via explicit matrix iterations:

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)} \quad (4)$$

$$a_j^{(l)} = f(z_j^{(l)}) \quad (5)$$

where w represents the synaptic weights and b the bias. A custom thresholding function $f(x)$ was implemented to strictly binarize the output layer, translating continuous network calculations into distinct mechanical key-presses for the physics engine.

B. Genetic Algorithm Mechanics

The optimization is driven by a custom Genetic Algorithm. A population of $N = 100$ distinct neural agents is instantiated per generation with randomized weights $W \in [-1, 1]$. As agents navigate, a continuous *Fitness Function* evaluates their success. To encourage speed and safety, fitness F was defined as a function of forward distance d penalized by collisions:

$$F = \alpha \cdot d_{\text{forward}} - \beta \cdot (\text{Collision Penalty}) \quad (6)$$

The algorithm utilizes strict *Elitism*, preserving the synaptic weights of the highest-scoring agent. To spawn the next generation, this elite genome is mutated using a linear interpolation function combined with a stochastic mutation rate μ :

$$W_{\text{new}} = \text{Lerp}(W_{\text{best}}, \text{Random}(-1, 1), \mu) \quad (7)$$

V. IMPLEMENTATION & VISUAL RESULTS

A. End-to-End Simulation Interface

The culmination of this from-scratch engineering effort is a highly performant, real-time software application. The interface renders the physical environment, the sensor raycasting, and the dynamic traffic generation.

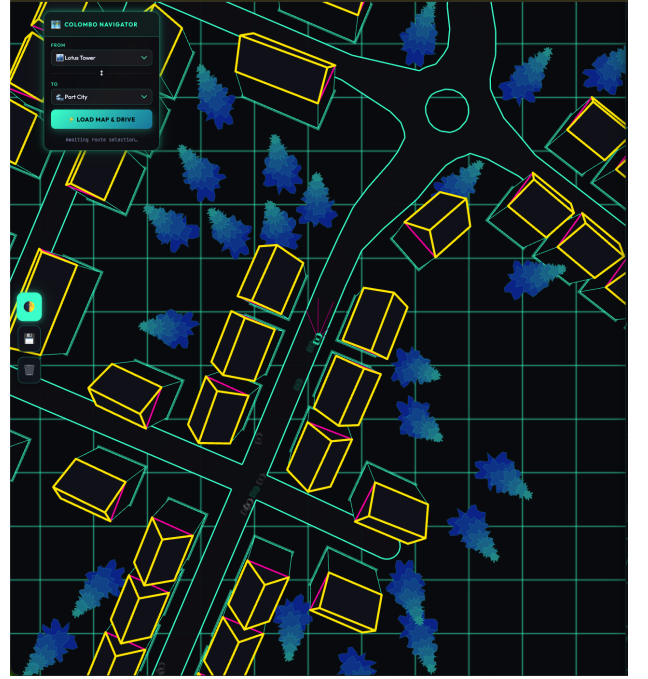


Figure 1. Overview of the custom-built simulator environment. The primary agent (bottom) uses 5 LiDAR rays to detect boundaries and slower traffic.

B. Real-time Neuro-Visualizer (Explainable AI)

To ensure the custom neural network is transparent, a real-time auditing dashboard was built. This visualizer maps the forward propagation frame-by-frame, demonstrating exactly which sensor inputs cross the threshold to trigger specific steering outputs, thereby providing a clear model for Explainable AI (XAI).

VI. EXPERIMENTAL OBSERVATIONS

The implementation proved highly successful, with the model demonstrating three distinct phases of learning over time:

- **Exploration Phase (Gen 1-15):** High mutation variance resulted in erratic behavior. Agents frequently collided with walls, mapping the boundaries of the action space and establishing baseline fitness scores.

- **Learning Phase (Gen 16-35):** The genetic algorithm isolated weight structures that promoted sustained forward movement. Agents developed a recognizable "balancing" strategy, utilizing lateral LiDAR sensors to stay centered within the lane.
- **Optimization Phase (Gen 50+):** The elite models achieved convergence. Agents consistently demonstrated the ability to execute high-speed maneuvers, weave through dynamic traffic obstacles without braking, and recover from near-collisions.

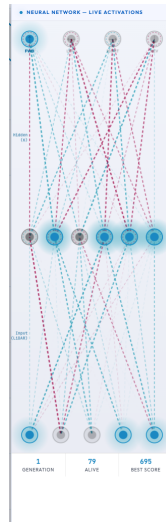


Figure 2. Visualizing synaptic firing. Active nodes and weighted connections are highlighted during complex obstacle avoidance maneuvers.

VII. SOFTWARE AVAILABILITY

The complete source code, containing the unabridged, from-scratch implementations of the physics engine, MLP, and GA, is available for technical review on GitHub:

https://github.com/DeegayuA/NN-Auto-Car_simulator

VIII. CONCLUSION

This capstone project successfully demonstrates the end-to-end synthesis of an autonomous agent without reliance on pre-existing AI frameworks or commercial game engines. By mathematically constructing a kinematic physics environment, deriving raycasting algorithms for spatial perception, and hard-coding the forward propagation and evolutionary optimization of a neural network, this project proved that complex, autonomous behaviors can be synthesized from ground-level programming logic. The resulting system effectively bridged theoretical machine learning concepts with applied algorithmic engineering, resulting in a highly robust navigational controller capable of dynamic obstacle avoidance.

ACKNOWLEDGMENTS

I would like to extend my sincere gratitude to the Main Examiner, Dr. Thanuja Ambegoda, and the broader evaluation panel for their guidance. The rigorous academic standards of the MDS&AI program deeply influenced the technical depth and "from-scratch" methodology of this final implementation.

REFERENCES

- [1] S. Arzt, "DeepCars: Neuroevolution in Unity," *ArztSamuel Projects*, [Online]. Available: <https://arztsamuel.github.io/en/projects/unity/deepCars/deepCars.html>.
- [2] R. Marescu-Istodor, "Self Driving Car Simulation," *Radu's Virtual World*, [Online]. Available: <https://radufromfinland.com/projects/virtualworld/>.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [4] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.

DECLARATION OF ORIGINALITY

I, Deeghayu Suwahas Adhikari (Index: 258722D), hereby declare that this capstone report and the accompanying code-base represent my own original work, implemented independently to fulfill the final assessment requirements of the In25-S3-CS5998 module.